



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

Tutorial 3 : AI Algorithm CNN

NAME	MATRIC NO
MUHAMMAD NAIM BIN ABDULLAH	A23CS0134
MUHAMMAD ADAM BIN RAZALI	A23CS0116
AFIF SHAQIR IRFAN BIN ARQAM	A23CS0204

What is Image Classification?

Image classification is supervised machine learning where an algorithm assigns a single class label to an input image based on its visual content. Mathematically, given an image and a set of 10 possible classes including airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The model learns a mapping to output the class with the highest probability

What is CNN?

A Convolutional Neural Network is a deep learning architecture designed for grid-structured data like 2D images. CNN optimize the visual processing via local receptive fields (localized analysis), shared weights (consistent filter application), and hierarchical feature composition (building complex shapes from simple edges). The Core Architecture Layer of the CNN model include:

- **Convolutional Layer (Conv2D)**
Slides learnable filters across the input to generate feature maps. Early layers detect basic edges and deeper layers for complex object parts.
- **Activation Function (ReLU)**
Replaces negative values with zero to introduce non-linearity enabling the network to learn complex and real world patterns.
- **Pooling Layer (MaxPooling2D)**
Down-samples feature maps by extracting the maximum value within a moving window for example 2 x 2 pixels. This reduces the data size, complexity and grant a degree of translation invariance
- **Dropout Layer**
Randomly deactivates a fraction of neurons during training. This acts as a regularizer to prevent overfitting.
- **Flatten**
Converts multi-dimensional feature maps into a 1D vector, preparing the data for the final classification layers

- Dense Layer and Softmax:

Connect all remaining features to calculate final class scores. The terminal layer uses Softmax to convert the scores into probabilities distribution over the 10 target classes.

Evaluation of CNN

To quantify a trained CNN's performance on unseen data, the following evaluation metrics and tools are utilized :

- **Accuracy**

The proportion of correctly classified samples out of the total train dataset. It can show the performance weaknesses if the dataset has class imbalance.

- **Loss**

The cost function is minimized by the optimizer during training. Lower value indicates better performance. Observing the gap between training and validation loss helps identify overfitting

- **Confusion Matrix**

A 10 x 10 table mapping true classes against predicted classes. The diagonal cell represent correct classification while off diagonal cells highlight the specific misclassifications between classes.

- **Training Curves**

Plotting accuracy and loss against epoch number reveals overfitting via the gap between training and validation metrics, and indicate whether extended training is required.

Steps hand on in Python

Based on the code from the `cnn_image_classification.ipynb`, the code is broken down into several stages to build and train a basic Convolutional Neural Network (CNN) as explained below:

Importing libraries

```
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
from tensorflow.keras.utils import to_categorical
```

Import "tensorflow.keras" could not be resolved from source
Import "tensorflow.keras.utils" could not be resolved

Figure 1. Importing necessary libraries

Based on the code block above, we mainly use Tensorflow with Keras to provide the necessary component for the machine learning model and dataset which is CIFAR-10.

Data Loading and Exploration

```
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

print('Train images shape:', x_train.shape)
print('Train labels shape:', y_train.shape)
print('Test images shape :', x_test.shape)
print('Test labels shape :', y_test.shape)
print('Pixel value range :', x_train.min(), '-', x_train.max())
```

Figure 2. Code Snippet for loading the CIFAR-10 dataset

The block of code above shows the loading of the dataset CIFAR-10 directly from Keras' built-in datasets module via the Tensorflow library. The dataset contains 50,000 train images and 10,000 test images, each labelled airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck which are stored inside the `class_names` variable.

Data Preprocessing

```
x_train = x_train.astype('float32') / 255.0
x_test  = x_test.astype('float32') / 255.0

y_train_cat = to_categorical(y_train, 10)
y_test_cat  = to_categorical(y_test, 10)

print('After normalization, pixel range:', x_train.min(), '-', x_train.max())
print('One-hot label shape (train):', y_train_cat.shape)
print('Example one-hot label:', y_train_cat[0])
```

Figure 3. Code snippet for normalising the dataset

Before feeding the data into CNN model, it needs to be normalized to a format that the model can learn from efficiently. For each pixel inside the image, it is rescaled from the original 0-255 range into the 0-1 range. This approach allows for small activations and stabilises gradient updates. Second, the integer class labels are converted into one-hot encoded vectors of length 10 using `to_categorical`. For example, the label 3 becomes `[0,0,0,1,0,0,0,0,0,0]`. This format is required because the model's output layer uses softmax with categorical_crossentropy loss, which compares probability distributions across all ten classes.

Building and Compiling the CNN model

```
model = models.Sequential([
    layers.Input(shape=(32, 32, 3)),

    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    layers.Flatten(),
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax'),
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy'],
)

model.summary()
```

Figure 4. Code Snippet for defining the baseline CNN model

Based on the code block, the CNN model is built from two logical parts which are two convolutional blocks followed by a classification head.

- The convolutional blocks via `layers.Conv2D` act as a feature extraction where it uses 32 filters of 3 x 3 pixels to detect for color gradient, and low level patterns. The second convolutional block double the filters to capture more abstract features. Following the blocks is a pooling layer via `MaxPooling2D` to halve the spatial resolution and a `Dropout(0.25)` layer that randomly disables a quarter of the feature maps during the training to prevent overfitting.
- A classification head convert the feature extraction into the class prediction. `Flatten()` reshapes the final 3-D feature map into a 1-D vector, `Dense(512)` mixes those features into a high-level representation, `Dropout(0.5)` provides stronger regularisation at this most-parameter-heavy layer, and the final `Dense(10, activation='softmax')` outputs a probability distribution over the ten CIFAR-10 classes.
- After the model initialization, it is compiles using the Adam Optimizer for its adaptive learning rate. Cross entropy is for multi-classification with one-hot labels.

The resulting CNN model contains the following number of parameters that can be train and improved to capture the image features:

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
conv2d_1 (Conv2D)	(None, 30, 30, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
dropout (Dropout)	(None, 15, 15, 32)	0
conv2d_2 (Conv2D)	(None, 15, 15, 64)	18,496
conv2d_3 (Conv2D)	(None, 13, 13, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
dropout_1 (Dropout)	(None, 6, 6, 64)	0
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 512)	1,180,160
dropout_2 (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 10)	5,130

Figure 5. CNN model parameter details

Model Training and Validation

```
EPOCHS = 10
BATCH = 64

history = model.fit(
    x_train, y_train_cat,
    epochs=EPOCHS,
    batch_size=BATCH,
    validation_data=(x_test, y_test_cat),
    verbose=1,
)
```

Figure 6. Code Snippet for training the CNN model

The block of code present the training loop that updates the model's performance to minimize the loss and increases accuracy. The model is trained for 10 epochs, meaning it is passed into the model for 10 times. The validation data parameter allow the model to test on unseen data at the end of each epochs to calculate the current loss and accuracy.

```
Epoch 1/10
782/782 ————— 58s 71ms/step - accuracy: 0.4222 - loss: 1.5777 - val_accuracy: 0.5597 - val_loss: 1.2245
Epoch 2/10
782/782 ————— 53s 68ms/step - accuracy: 0.5817 - loss: 1.1725 - val_accuracy: 0.6249 - val_loss: 1.0628
Epoch 3/10
782/782 ————— 53s 68ms/step - accuracy: 0.6449 - loss: 1.0027 - val_accuracy: 0.6795 - val_loss: 0.9113
Epoch 4/10
782/782 ————— 49s 63ms/step - accuracy: 0.6833 - loss: 0.8981 - val_accuracy: 0.7097 - val_loss: 0.8370
Epoch 5/10
782/782 ————— 49s 63ms/step - accuracy: 0.7113 - loss: 0.8226 - val_accuracy: 0.7332 - val_loss: 0.7587
Epoch 6/10
782/782 ————— 51s 65ms/step - accuracy: 0.7299 - loss: 0.7725 - val_accuracy: 0.7528 - val_loss: 0.7093
Epoch 7/10
782/782 ————— 48s 61ms/step - accuracy: 0.7479 - loss: 0.7243 - val_accuracy: 0.7569 - val_loss: 0.7025
Epoch 8/10
782/782 ————— 49s 62ms/step - accuracy: 0.7570 - loss: 0.6887 - val_accuracy: 0.7665 - val_loss: 0.6655
Epoch 9/10
782/782 ————— 61s 78ms/step - accuracy: 0.7671 - loss: 0.6608 - val_accuracy: 0.7674 - val_loss: 0.6745
Epoch 10/10
782/782 ————— 58s 75ms/step - accuracy: 0.7786 - loss: 0.6267 - val_accuracy: 0.7790 - val_loss: 0.6535
```

Figure 7. Output of model training per epoch

This image shows an AI model successfully learning a task over 10 rounds (epochs). With each passing round, the model makes fewer mistakes and gets significantly better at guessing correctly, steadily improving its "accuracy" from about 42% in the first round to nearly 78% by the end. Most importantly, the model is performing just as well on fresh, unseen test data, indicated by the rising validation accuracy and dropping validation loss which means it is genuinely learning the underlying patterns rather than just memorizing the training material.

What is the weakest in the current CNN model?

Based on the training summarized in Table 2, the baseline model achieves a final test accuracy of 77.53%. Through the inspection of training behaviour and confusion patterns highlight several weaknesses including:

- Mild overfitting. From epoch 6 onwards, validation accuracy stagnates between 75% and 78% while training accuracy keeps increasing. The gap of validation and training accuracy keeps increasing indicating signs of overfitting.
- No data augmentation. The model sees the image in exactly same orientation, scale, and lighting. CIFAR-10 contains substantial real-world variation that the model never gets to practise on.
- Fixed learning rate. By using the Adam optimizer, it keeps the learning size the same and unable to take smaller more careful steps as the loss changes getting smaller.
- Imbalance class performance. From qualitative inspection of the predictions and the confusion matrix produced in the notebook, animal classes often get confused more than vehicle classes because of shared visual features among them.

How to enhance the CNN model

The baseline CNN got 77.53% accuracy after 10 epochs of training, but it had a few clear problems. It started to overfit from epoch 6, it always saw images in the same orientation and scale, and it used a fixed learning rate throughout. The new_CNN was built to fix all of these issues.

- **Data Augmentation**

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip('horizontal'),
    layers.RandomTranslation(0.1, 0.1),
    layers.RandomRotation(0.04), # ~14 degrees
    layers.RandomZoom(0.1),
], name='data_augmentation')

# Visualise the effect of augmentation on one sample
sample = x_train[7:8]
plt.figure(figsize=(10, 3))
for i in range(8):
    aug = data_augmentation(sample, training=True).numpy()[0]
    plt.subplot(1, 8, i + 1)
    plt.xticks([]); plt.yticks([])
    plt.imshow(np.clip(aug, 0, 1))
plt.suptitle('Eight random augmentations of the same training image')
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, '10_augmentation_examples.png'), dpi=120)
plt.show()
```

Figure 8. Code Snippet for Data Augmentation on new_CNN Model

The most critical weakness of the baseline was that it trained on static images with no variation in orientation, scale, or lighting. The new_CNN introduces a data augmentation sub-model applied during training only, consisting of four stochastic transformations: RandomFlip (horizontal), RandomTranslation ($\pm 10\%$), RandomRotation (approximately $\pm 14^\circ$), and RandomZoom ($\pm 10\%$). By synthetically diversifying the training distribution, the model learns features that are invariant to these real-world transformations, directly reducing overfitting and improving generalisation on the CIFAR-10 test set.

An example of random augmentations of the same image is shown as below:

Eight random augmentations of the same training image



Figure 9. Data Augmentation Output on similar image

```
def build_new_cnn():
    return models.Sequential([
        layers.Input(shape=(32, 32, 3)),
        data_augmentation,

        # Block 1
        layers.Conv2D(32, 3, padding='same'), layers.BatchNormalization(), layers.Activation('relu'),
        layers.Conv2D(32, 3, padding='same'), layers.BatchNormalization(), layers.Activation('relu'),
        layers.MaxPooling2D(2),
        layers.Dropout(0.2),

        # Block 2
        layers.Conv2D(64, 3, padding='same'), layers.BatchNormalization(), layers.Activation('relu'),
        layers.Conv2D(64, 3, padding='same'), layers.BatchNormalization(), layers.Activation('relu'),
        layers.MaxPooling2D(2),
        layers.Dropout(0.3),

        # Block 3
        layers.Conv2D(128, 3, padding='same'), layers.BatchNormalization(), layers.Activation('relu'),
        layers.Conv2D(128, 3, padding='same'), layers.BatchNormalization(), layers.Activation('relu'),
        layers.MaxPooling2D(2),
        layers.Dropout(0.4),

        # Head
        layers.GlobalAveragePooling2D(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(10, activation='softmax'),
    ], name='new_cnn')

new_cnn = build_new_cnn()
new_cnn.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss='categorical_crossentropy',
    metrics=['accuracy'],
)
new_cnn.summary()
```

Figure 10. Code Snippet for new_CNN model initialization

- **Deeper Convolutional Architecture (Three Blocks)**

The baseline used two convolutional blocks with 32 and 64 filters respectively. The new_CNN expands this to three blocks with a $32 \rightarrow 64 \rightarrow 128$ filter progression. Each block follows the pattern Conv2D $\rightarrow$ BatchNorm $\rightarrow$ ReLU (applied twice), followed by MaxPooling2D and Dropout. The third block with 128 filters enables the model to capture higher-level abstract representations such as compound object parts, which is particularly important for visually similar animal classes.

- **Batch Normalisation**

Batch Normalisation (BatchNorm) is inserted after every Conv2D layer, before the ReLU activation. This normalises the output of each convolutional layer to have zero mean and unit variance across the mini-batch, which stabilises gradient flow and enables training with higher learning rates. BatchNorm also acts as a regulariser, reducing the model's sensitivity to weight initialisation and mitigating internal covariate shift. In combination with data augmentation, it significantly reduces the overfitting gap observed in the baseline.

What is the evaluation based on CNN and the new_CNN model?

- **Overall Accuracy and Loss**

Both models were tested on the same 10,000-image CIFAR-10 test set after being trained on the same 50,000-image training set. Both used TensorFlow 2.21.0 and ran on CPU only, so the comparison is fair. The evaluation looks at overall accuracy and loss, how training went over time, signs of overfitting, the confusion matrix, and how each model performed on individual classes.

Metric	Baseline CNN	new_CNN	Delta
Test Accuracy	77.53%	80.45%	+2.92pp
Test Loss	0.6535	0.5758	-0.0777
Total Parameters	1,251,858	306,602	-75.5%
Epochs Trained	10	14 (of 30)	Early stopped
Overfitting Sign	Yes (Epoch 6+)	Minimal	Improved

Table 1 : Comparative Performance Metrics : Baseline CNN vs new_CNN

The new_CNN was set to run for up to 30 epochs, but EarlyStopping ended training at epoch 14 and loaded back the best weights from epoch 13. This shows that BatchNorm and data augmentation were effective enough that the model did not need to train for nearly as long as the baseline to get better results.

- **Training Curves and Overfitting Analysis**

Figure 11 shows the training curves for new_CNN on its own. The model starts slower than the baseline its validation accuracy at epoch 1 was only about 33%, compared to 55.97% for the baseline. This is because data augmentation makes each training step harder, since the model sees a different version of each image every time. However, the accuracy then climbs steadily and reaches 80.45% at epoch 13.

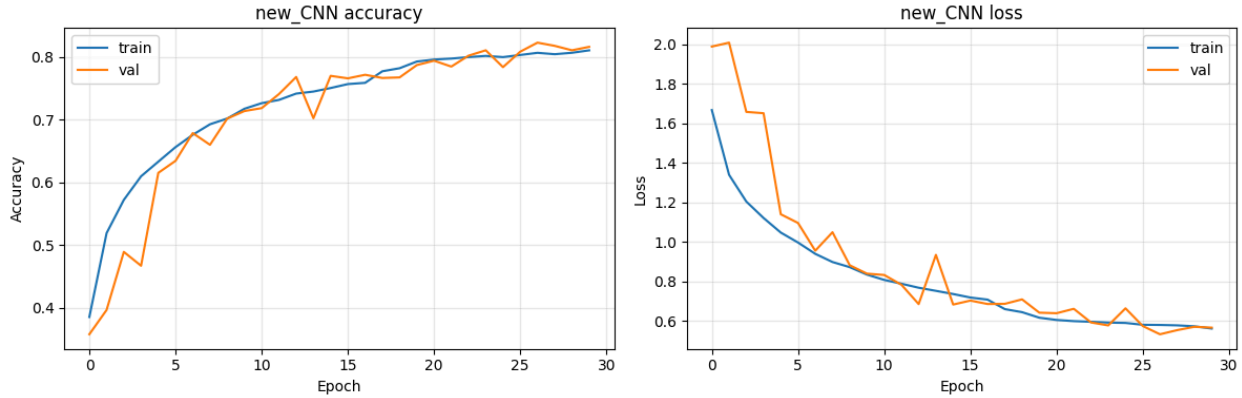


Figure 11. plot for new_CNN accuracy and new_CNN loss

Two learning rate reductions can be seen in Figure 3. At epoch 8, ReduceLROnPlateau cut the learning rate from 0.001 to 0.0005 after detecting that validation loss had stopped improving. This caused a noticeable jump in accuracy at epochs 9 and 10. A second cut at epoch 12 (to 0.00025) led to another improvement at epoch 13, which turned out to be the best checkpoint.

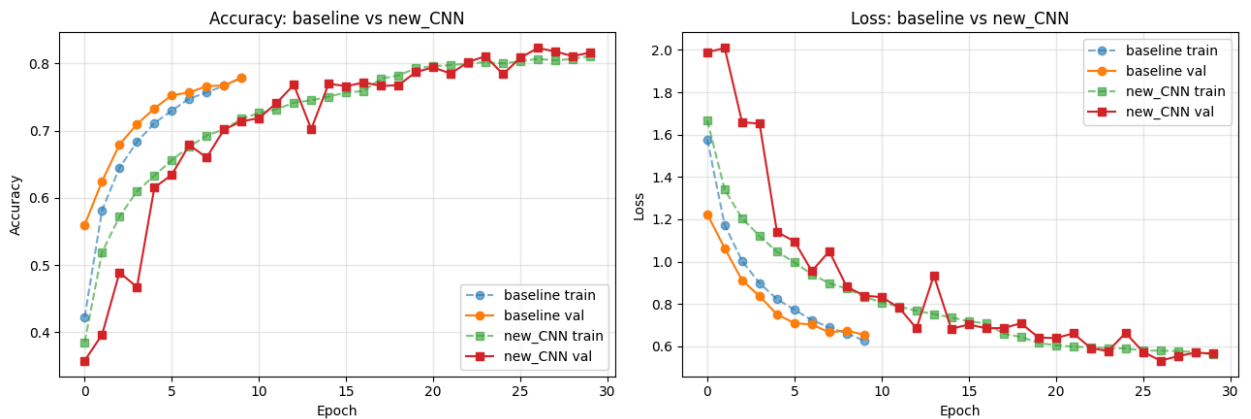


Figure 12. Side-by-Side Training Curves — Baseline CNN vs new_CNN

Figure 12 puts the two models side by side. The baseline model shows a clear split between training and validation accuracy starting from epoch 6 training accuracy keeps going up while validation accuracy levels off. This is a sign of overfitting. The new_CNN does not show this problem. Its training and validation curves stay close together across all 14 epochs, which means the model is genuinely learning from the data rather than just memorising the training set.

- Confusion Matrix Analysis**

Figure 13 shows the confusion matrices for both models. Each row is a true class and each column is what the model predicted. The numbers on the diagonal are correct predictions, and everything else is a mistake. In both models, vehicle classes like automobile, ship, and truck are predicted well, while animal classes like cat, dog, deer, and bird are harder to get right because they can look similar to each other.

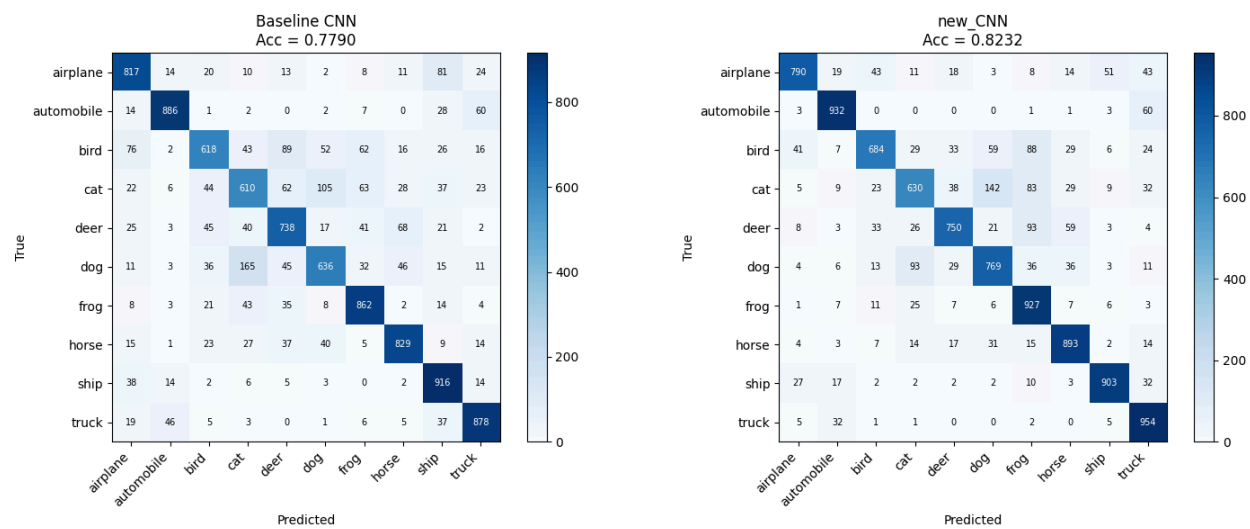


Figure 13. plot for predicted of baseline CNN vs new_CNN

Looking at the new_CNN matrix, most classes improved compared to the baseline. Automobiles went from 896 to 932 correct predictions out of 1,000, and trucks improved from 878 to 954. Cat and dog are still the toughest classes for both models since they share many similar visual features, but the new_CNN does make fewer cat-dog mix-up errors than the baseline.

- **Per-Class Accuracy Comparison**

Figure 14 show how each of the ten classes performed individually. The new_CNN improved on most classes, with the biggest gains in truck (+7.6 pp), frog (+6.5 pp), and automobile (+3.6 pp). These are classes with clear and distinct visual features, so the deeper architecture and better training were able to pick up on those features more reliably.

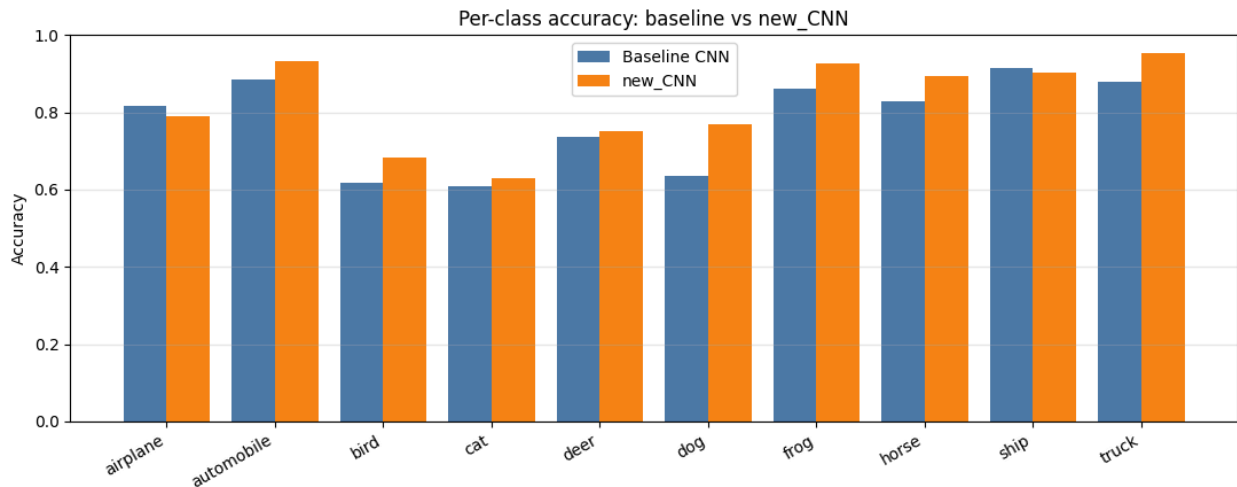


Figure 14. plot for predicted of baseline CNN vs new_CNN

The one class that got slightly worse was airplane (-6.7 pp). This is likely because the horizontal flip augmentation confuses the model when classifying planes, since flipping a plane left to right changes how it looks in a meaningful way. This could be fixed in future work by turning off the horizontal flip for classes where orientation matters. All other classes were the same or better with the new model.

Results and Discussion

This tutorial built and compared two CNN models for classifying images from the CIFAR-10 dataset: a baseline CNN and an improved version called new_CNN. Both were trained and tested under the same conditions using TensorFlow 2.21.0 on CPU, with the same training and test images.

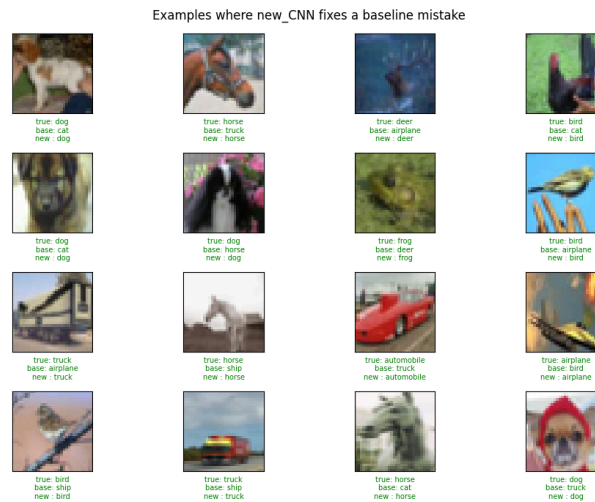


Figure 15. Image of fixed baseline mistake by new_CNN

• Baseline CNN Performance

The baseline CNN reached 77.53% accuracy after 10 epochs. It started at 42.22% in epoch 1 and gradually improved to 77.86% by epoch 10. For a simple two-block model with no data augmentation, this is a reasonable result.

However, looking at the training curves and confusion matrix, some clear problems stand out. From epoch 6 onwards, the training accuracy kept going up while validation accuracy stopped improving and stayed between 75% and 78%. This means the model was starting to overfit — it was memorising the training data instead of learning patterns that work on new images. The Dense(512) layer alone had 1.18 million parameters, making up 94% of all the model's parameters, but only had one Dropout(0.5) layer to control overfitting. On top of that, the model always saw the same images in the same position, so it never learned to handle variation in real-world photos.

• **Enhanced new_CNN Performance**

The new_CNN was built to fix these problems. Based on the training logs from new_cnn_comparison.ipynb, the model starts slower than the baseline because its validation accuracy at epoch 1 was around 33% because data augmentation makes training harder. But from there, the accuracy grows in a steady and stable way throughout training.

By epoch 13, validation accuracy reached 80.45%, and EarlyStopping ended training at epoch 14 using those best weights. The learning rate was cut in half at epoch 8 and again at epoch 12 by ReduceLROnPlateau, which caused visible accuracy jumps each time. Most importantly, the training and validation accuracy stayed close together the whole time, with no sign of the overfitting that appeared in the baseline.

The new_CNN uses 306,602 parameters compared to 1,251,858 in the baseline with a 75.5% drop. Getting better accuracy with far fewer parameters shows that adding BatchNorm, data augmentation, and Global Average Pooling was a much smarter approach than just making the dense layers bigger.

• **Key Findings**

The main takeaways from comparing both models are:

- Data augmentation had the biggest impact. It stopped the overfitting problem and helped the model learn features that work across different image orientations and scales.
- Batch Normalisation kept the training stable and also helped the model generalise better, working alongside the Dropout layers.
- Global Average Pooling removed the need for the large Dense(512) layer and cut the parameter count dramatically while still improving accuracy.
- The adaptive learning rate helped squeeze out extra accuracy by letting the model make finer adjustments as training progressed.
- Most classes improved, with the biggest gains in truck, frog, and automobile. Cat and dog were still the hardest for both models since they look so similar.

- **Limitations and Future Work**

There are still some limitations to address. Both models were trained on CPU only, which made training slow and limited how many epochs could be run. The airplane class got worse with the new model, likely because flipping airplane images sideways during augmentation is not realistic and confuses the model. A fix would be to turn off horizontal flip specifically for the airplane class.

For future improvements, using a pre-trained model like ResNet or EfficientNet could push accuracy much higher with less training time. Other techniques like mixup augmentation or cosine learning rate schedules are also worth trying.

Reflection

MUHAMMAD NAIM BIN ABDULLAH

This tutorial successfully demonstrated how to build and analyze an image classification model using a Convolutional Neural Network (CNN). Beyond implementing the code, a new knowledge was gained into how convolutional layers extract visual patterns and how evaluation tools like loss & accuracy curves and confusion matrices are critical for accurately diagnosing model performance. Developing a strong comprehension of these deep learning and computer vision fundamentals is highly valuable, providing an essential skill set for future professional growth in data science and automated technologies.

MUHAMMAD ADAM BIN RAZALI

Throughout the tutorial, the most valuable lessons is the capability to identify how image classification is done using the basic CNN model derived from the ANN model. The more challenging part was identifying the weaknesses of the baseline CNN model that include deciding for more neuron or more filters. However, after analysing the confusion matrix and the training curves, learning that the model was overfitted and there were class imbalances. Addressing these with batch normalisation, data augmentation, and global average pooling produced a model that was both smaller and accurate.

AFIF SHAQIR IRFAN BIN ARQAM

This tutorial showed how image classification works using CNN, going from a simple baseline model to an improved version. By looking at the confusion matrices and training curves, I was able to spot the problems in the baseline because mainly overfitting and the lack of image variation during training. Fixing these with data augmentation, batch normalisation, and global average pooling led to a smaller and more accurate model. This was a good hands-on experience that helped me understand how to diagnose model problems and apply practical fixes, which I think will be useful in future data science work.